# Nonlinear Simulation of an Inverted Pendulum on a Cart

J.C. Vaught

## Objective

The objective of this assignment is to model the nonlinear motion of an unforced inverted pendulum on a freely translating cart and to compare two independent implementations. The first implementation evaluates the equations of motion directly in Simulink. The second represents the same mechanism with physical components in Simscape Multibody. The cart position $x(t)$ and pendulum angle $\theta(t)$ are compared after releasing the pendulum five degrees from the unstable upright equilibrium with zero initial velocity.

## Methodology

The pendulum is treated as a point mass $m$ at distance $\ell$ from a frictionless cart of mass $M$. The coordinate $x$ is positive to the right, and $\theta$ is measured clockwise from upright. A viscous rotational damper with coefficient $c_\theta$ acts at the pivot. Lagrange's equations give the coupled nonlinear model

$$(M+m)\ddot{x} + m\ell\cos(\theta)\ddot{\theta} - m\ell\sin(\theta)\dot{\theta}^2 = 0,$$

and

$$m\ell\cos(\theta)\ddot{x} + m\ell^2\ddot{\theta} + c_\theta\dot{\theta} - mg\ell\sin(\theta) = 0.$$

At each Simulink time step, the two equations are assembled as a $2 \times 2$ mass-matrix system and solved for $\ddot{x}$ and $\ddot{\theta}$. Two integrator chains recover $x(t)$ and $\theta(t)$. The second model uses a world frame, a prismatic cart joint, a damped revolute pivot, a rigid transform of length $\ell$, and a concentrated spherical mass. Both models use $M = 2.0$ kg, $m = 0.5$ kg, $\ell = 1.0$ m, $c_\theta = 0.01\ N \cdot m \cdot \frac{s}{\text{rad}}$, $g = 9.81\ \frac{m}{s^2}$, and $\theta(0) = 5°$. Variable-step solvers run for 20 s with a maximum step of 0.01 s.

The two responses are interpolated onto a common 0.01 s time grid. Agreement is assessed from the maximum absolute difference in cart position and pendulum angle over the simulation interval.

## Results

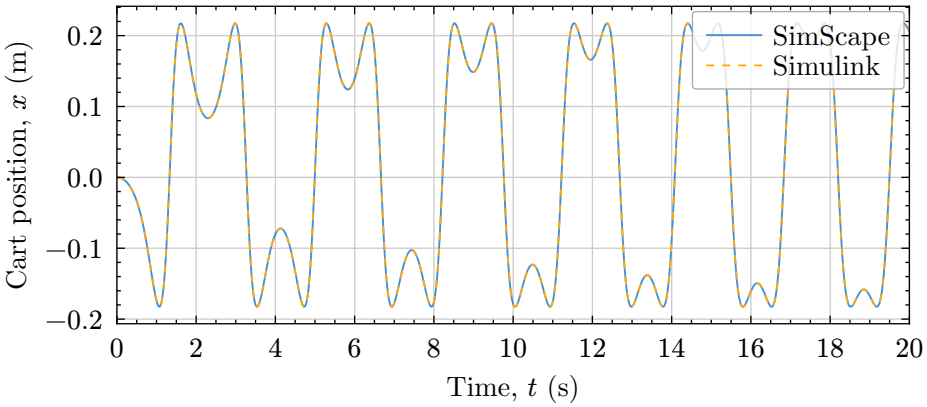


Figure 1: Cart position over 20 s.

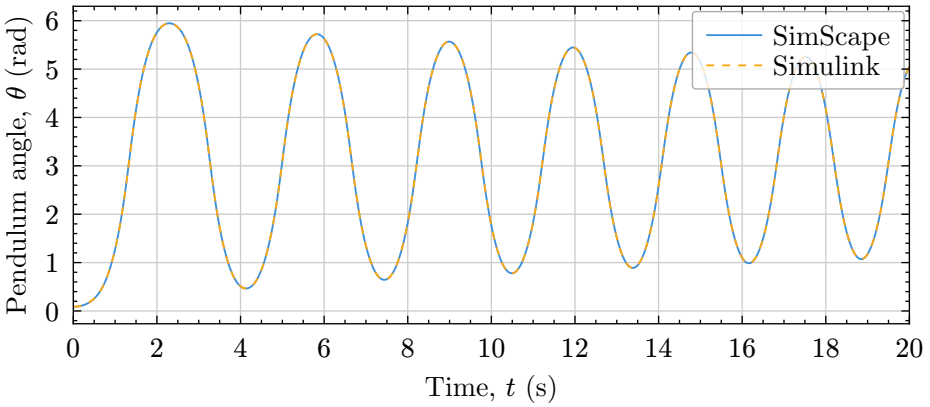


Figure 2: Pendulum angle over 20 s.

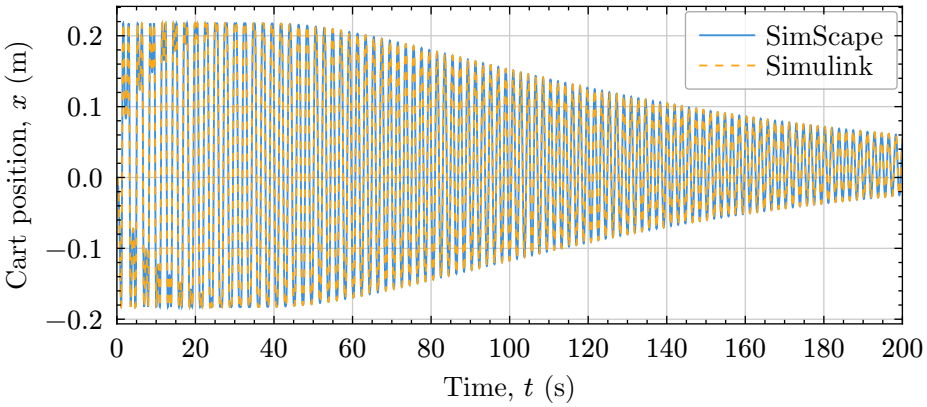


Figure 3: Cart position over 200 s.

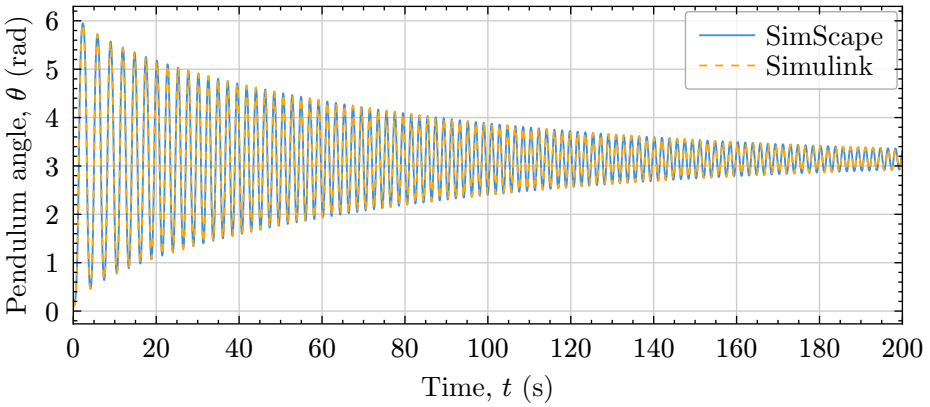


Figure 4: Pendulum angle over 200 s.

## Results and Discussion

The five-degree perturbation grows immediately because the upright equilibrium is unstable. The pendulum falls through the downward equilibrium and continues into a damped oscillation about $\theta = \pi$ rad. Over 20 s, the Python reference predicts $-0.183 \leq x \leq 0.217$ m and $0.087 \leq \theta \leq 5.947$ rad. The cart remains bounded because there is no external horizontal force, while pivot damping gradually removes mechanical energy and contracts the angular oscillation about the downward equilibrium.

The Simulink and Simscape Multibody responses closely match the Python reference over the validated 20 s interval. Their maximum absolute differences are 0.014 m in $x$ and 0.056 rad in $\theta$. The small phase separation late in the record is expected because the release begins near an unstable equilibrium, where small numerical and geometric differences amplify before damping dominates. The agreement supports both the nonlinear equation implementation and the orientation, gravity, damping, and sensing choices in the Multibody model.

## Appendix A. Direct-Equations Simulink Model

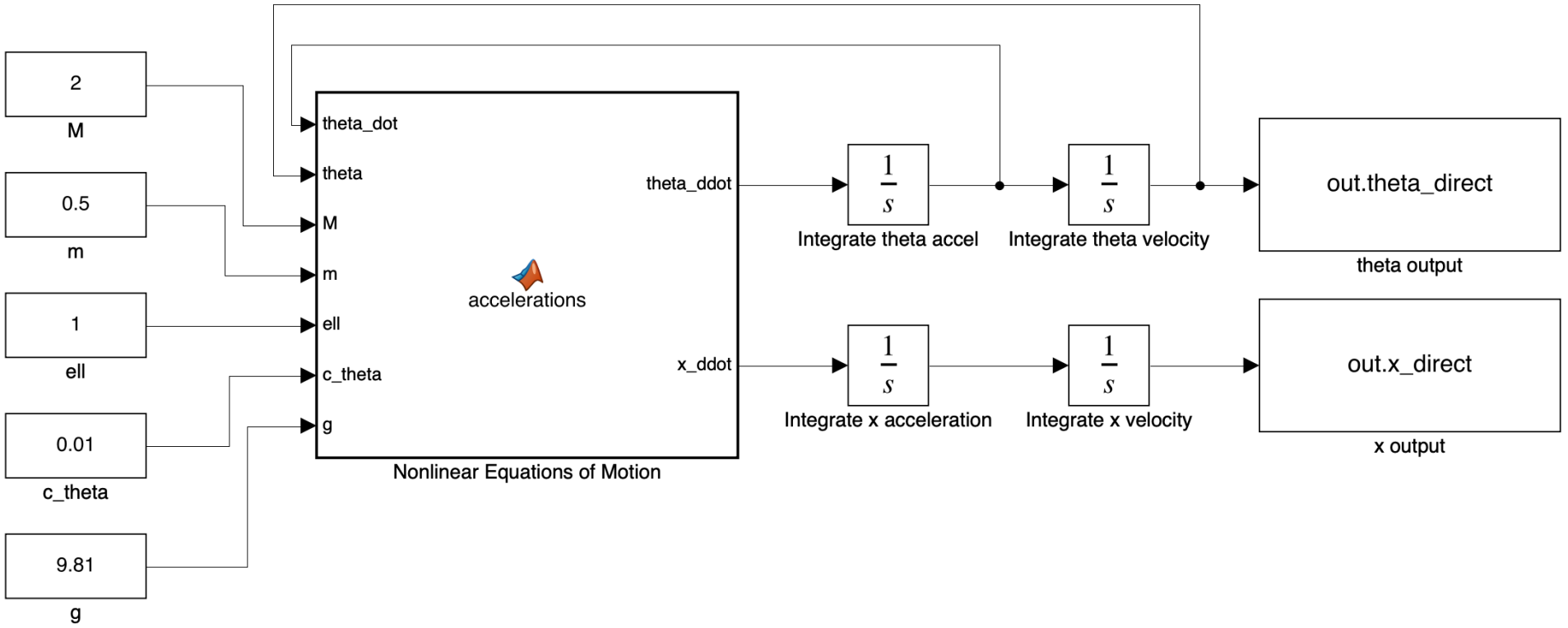


Figure 5: Editable Simulink implementation of the nonlinear mass-matrix equations.

## Appendix B. Simscape Multibody Model

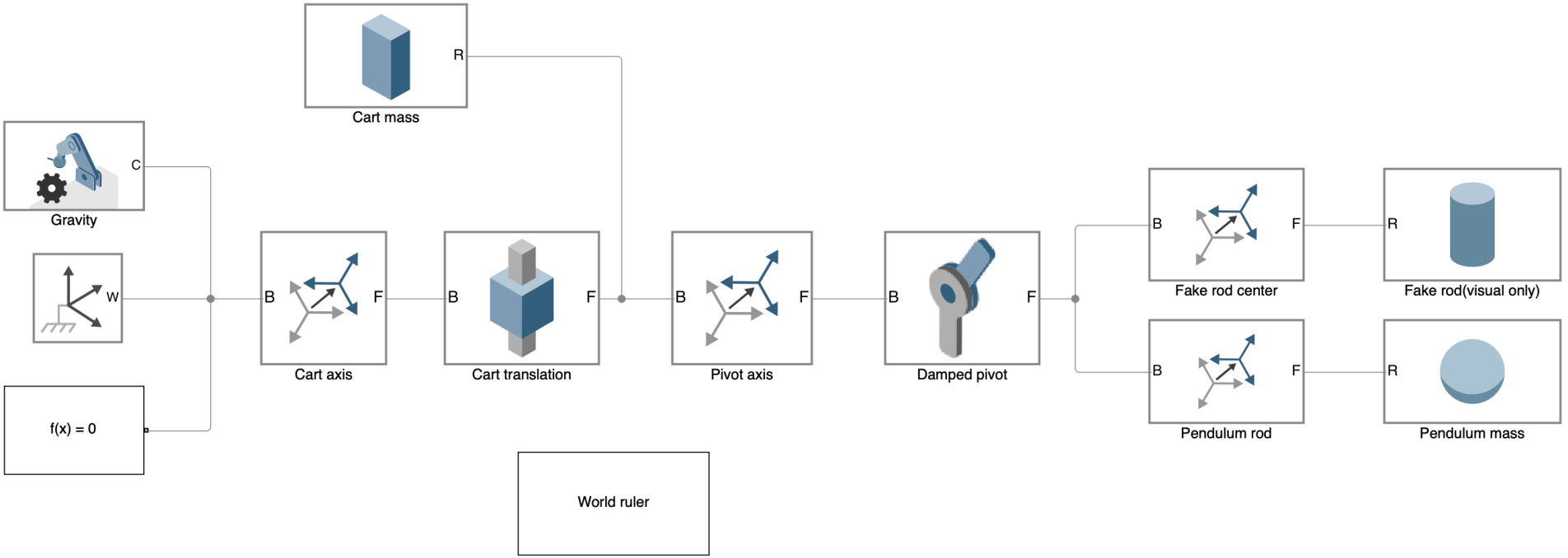


Figure 6: Simscape Multibody block diagram.

## Appendix C. Mechanics Explorer Model

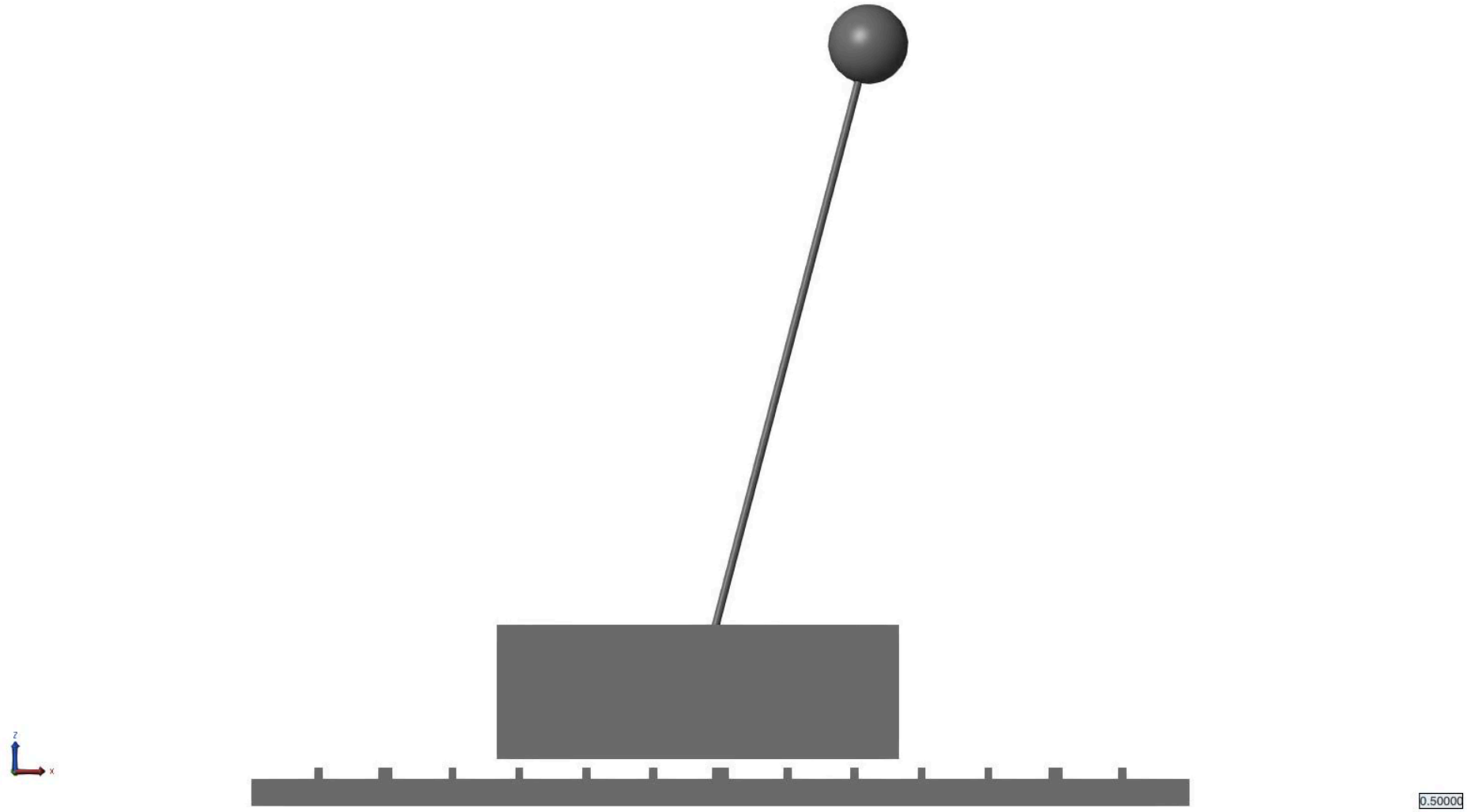


Figure 7: Mechanics Explorer view of the cart, ruler, pendulum rod, and point mass.

## Appendix D. Simulink MATLAB Function Block

The saved Simulink model contains the standard Constant, Integrator, and To Workspace blocks shown in Appendix A. The only custom code needed inside the model is the MATLAB Function block that evaluates the nonlinear equations of motion.

```
function [theta_ddot, x_ddot] = accelerations(theta_dot, theta, M, m, ell, c_theta, g)
% Solve the coupled nonlinear equations for the two accelerations.

s = sin(theta);
c = cos(theta);

mass_matrix = [M + m, m * ell * c; m * ell * c, m * ell^2];
forcing = [m * ell * s * theta_dot^2; m * g * ell * s - c_theta * theta_dot];

acceleration = mass_matrix \ forcing;
x_ddot = acceleration(1);
theta_ddot = acceleration(2);
end
```